— Exploring Batch Repetition to Reduce Data IO —

Anthony Fuller
Department of Systems and Computer Engineering
Carleton University
Ottawa, Canada K1S 5B6
anthonyfuller@cmail.carleton.ca

December 19, 2022

Abstract

Vast computational resources are required to develop high-performing deep neural networks (DNNs); training a state-of-the-art DNN on a *single* device—even the most powerful deep learning accelerator—would take months or even years. To rectify this, researchers and practitioners train DNNs on multiple devices in parallel by averaging gradients across devices. In this work, I speed up training by 2.7 times by training a remote sensing vision transformer on an 8-core TPUv3 machine, compared with an RTX 3090 GPU. But with faster or more accelerators, loading data from storage can become a bottleneck. To rectify this, I leverage batch repetition to speed up DNN training while preserving model accuracy.

1 Introduction

Deep neural networks (DNN) gradually transform input features—predominantly by multiplying them with millions of learned parameters over a series of layers—to produce predictions. Often correlated with the number of parameters, the more transformations a DNN can make, the more complex predictions it can make. But performing these transformations sequentially would take far too long.

Researchers started to leverage multi-core CPUs and GPUs in the 2000s [1, 2, 3, 4, 5, 6] to train shallow neural networks. But it wasn't until Krizhevsky et al. [7] leveraged GPUs to train DNNs in 2012 that DNNs became the dominant paradigm in computer vision. Krizhevsky's use of GPUs allowed them to train a convolutional neural network [8] (CNN) on 1.2 million images for 40 epochs, lasting 5-6 days. Buried in their paper, they write, "All of our experiments suggest that our results can be improved simply by waiting for faster GPUs and bigger datasets to become available"; ten years later, this observation holds. Many large-scale studies have demonstrated that increasing the amount of training data, model parameters and computation reliably improves the accuracy of DNNs—even scaling to billions of images and model parameters [9, 10, 11, 12].

Although most DNN training is performed on GPUs, GPUs are not designed specifically for this purpose; Tensor Processing Units [13, 14] (TPUs) are explicitly designed for DNN training. The primary difference between GPUs and TPUs is the size of the matrix multiplication units (MXU); 128x128 for TPUs versus 4x4 (or 4x8) for modern GPUs. These larger MXUs allow TPUs to be more efficient than GPUs for DNN training.

Although AI accelerators (i.e., GPUs or TPUs) have significantly improved over the past 5 years, they are not the only component in a DNN training pipeline. Data preparation (e.g., loading data from storage, preprocessing, data augmentation) is the other main component and is typically handled by the host CPU. Historically in DNN training, data preparation time has been negligible compared with accelerator time, but faster accelerators operating on more data have created a data IO bottleneck (i.e., where loading data from storage substantially slows down DNN training wall-clock time) for many researchers. Batch repetition [21] (BR) is a method that reuses a batch of data loaded from storage multiple times before loading the next batch. This method improves accelerator utilization—easing the data IO bottleneck—and is seeing increasing use across many applications.

In this work, I intend to leverage (i) multi-core TPUs to speedup DNN training, and (ii) BR to speedup DNN training by reducing data IO.

2 Literature Review and Background

2.1 Basics of Model Training

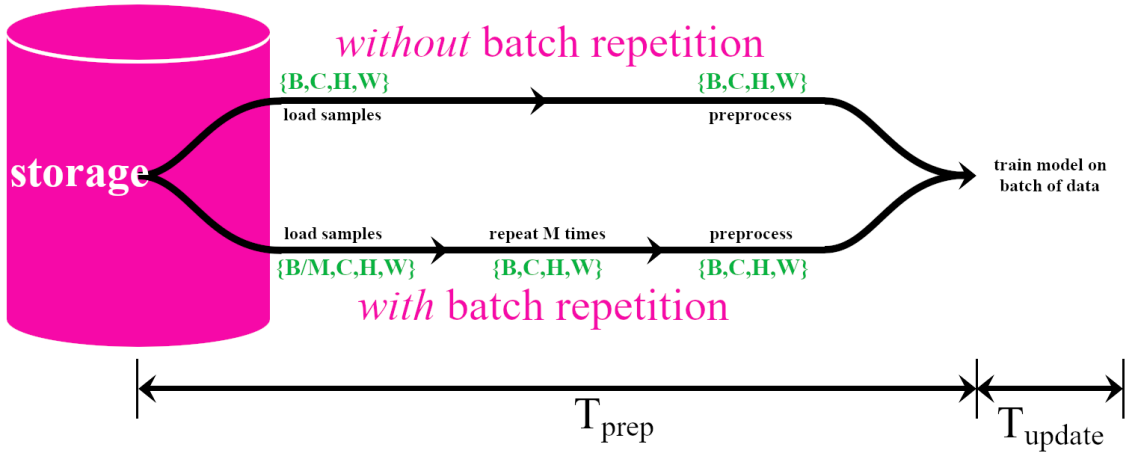

Figure 1: Model training with batch repetition loads fewer samples (by a factor of M) from storage (reducing data IO), then repeats the batch M times with data augmentation. B is batch size, M is batch repetition factor, C is channels, H is height, and W is width.

The millions of parameters in a DNN are slowly adjusted (starting from a random initialization) throughout training to learn a function that maps inputs to predictions. For our purposes, we split up a DNN training pipeline into data *preparation* tasks (typically handled by a CPU) and model *update* tasks (typically handled by an AI accelerator).

The host CPU first loads training data from storage, then prepares the data for the DNN. In some scenarios, preparation involves simply moving the data to the accelerator; in others, it involves complex normalization and augmentation methods before moving the data to the accelerator—these are data preparation tasks. Next, the data is processed by a forward-pass through the DNN, the loss value is calculated, then in a backward-pass through the DNN (to calculate gradients), and finally the model parameters are updated based on the gradients—these are model update tasks.

The actual speedup factor—due to leveraging BR—is a function of: model architecture, model size, batch size, device hardware, device algorithms, data, file systems, parallel data

loading, storage, CPU speed, etc. In order to make our findings insightful for all applications and infrastructures—rather than my precise application and infrastructure—I will only report speedup factors as a function of accelerator utilization. This refers to the ratio of the time it takes to perform model tasks to all tasks. Formally,

$$util = \frac{T_{update}}{T_{update} + T_{prep}} \tag{1}$$

For the vast majority of deep learning settings, data normalization, augmentation, and transferring data to the device is *very* fast—making these tasks negligible compared to data IO and model updating. Thus, we can say that T_{prep} approximately equals data IO time. As a result, repeating our batch M times reduces T_{prep} by M times. And our speedup due to BR is:

$$speedup = \frac{T_{prep} + T_{update}}{T_{prep}/M + T_{update}} = \frac{1}{(1 - util)/M + util} \tag{2}$$

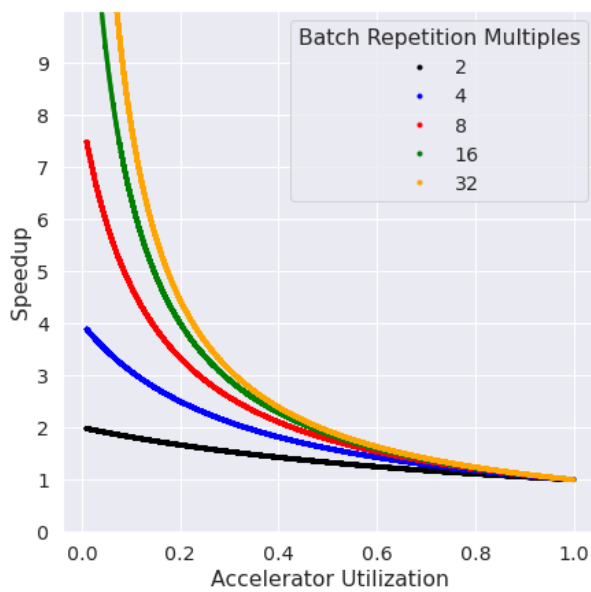

Figure 2: Speedup gains due to BR

Figure 2 plots the speedups—swept across M {2, 4, 8, 16, 32}—using equation 2. This plot can help us understand for how BR and utilization interact to yield speedups. We can see that low utilization benefits tremendously from BR and that increasing M yields diminishing returns. Although the best speedups occur at low utilization, higher utilization can still benefit from BR—otherwise, Facebook AI would not have used this technique, despite their near-perfect infrastructure [30].

2.2 Basics of Vision Transformers and Masked Autoencoding

Introduced in 2017, a transformer [15] is a DNN architecture that is leveraged in most state-of-the-art AI systems. A transformer adapted for computer vision tasks is called a vision transformer [16] (ViT) and is fundamentally different from a CNN. ViTs process spatial information via the attention mechanism, whereas CNNs process spatial information via

the convolution mechanism. A ViT first splits an image (e.g., of shape {3, 256, 256} with dimensions {channels, height, width}) into patches (e.g., of shape {3, 8, 8}), then processes the patches, transforming them into encoded patch representations.

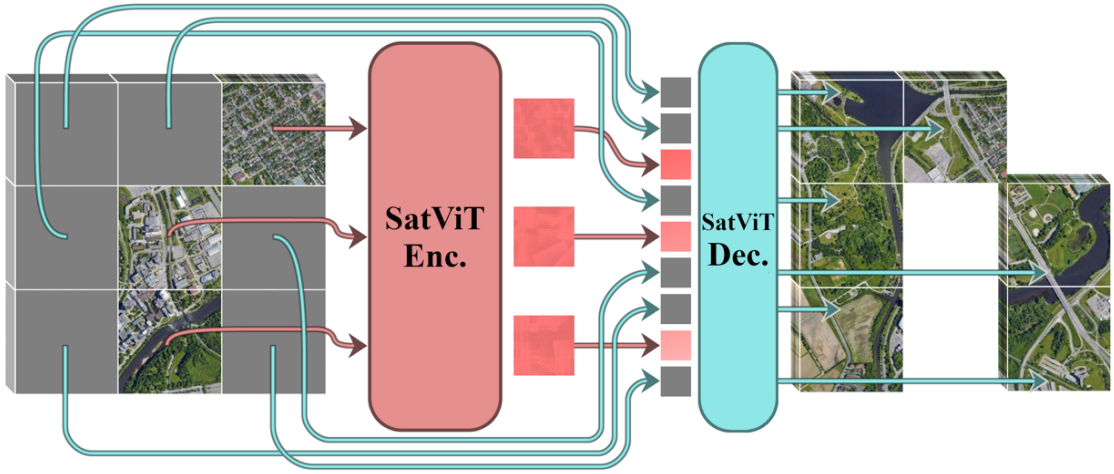

Figure 3: Masked autoencoding applied to remote sensing imagery. The size of patches are greatly exaggerated for better visualization of the algorithm. This figure is taken from Fuller et al. [18]

Masked autoencoding [28] (MAE) is a self-supervised learning algorithm that achieves state-of-the-art performance on the ImageNet [17] benchmark, but has also been leveraged in remote sensing [18], 3D point cloud [19], video processing [20], and more. MAE trains models by hiding a large portion (often 75%) of patches, encodes visible patches (with a ViT encoder), then uses these encoded visible patches to reconstruct hidden patches (with a ViT decoder)—model parameters are updated based on the reconstruction loss.

2.3 Batch Repetition Literature Review

BR was introduced by Hoffer et al. [21]; they demonstrated that BR could reduce data IO while slightly improving performance (measured by test accuracy) on several deep learning tasks. In particular, they leveraged convolutional neural networks (CNNs) trained via supervised learning on the ImageNet benchmark. They explored BR factors (M) {1, 2, 4, 8, 16, 32} and found that M-values greater than 4 performed similarly and outperformed the baseline (i.e., M=1). Building off this work, Fort et al. [22], trained CNN-based models via supervised learning on ImageNet. They explored M {1, 2, 4, 8, 16} and also found that higher M-values outperformed the baseline. Additionally, they showed that for a fixed number of effective epochs (i.e., the number of times each image is loaded from storage), higher M-values always resulted in better test accuracy. However, these two important studies focus on leveraging BR to improve the accuracy of deep learning models. In this work, we focus on leveraging BR to speedup model training.

Focused on speeding up model training, Choi et al. [23] demonstrated that BR could speedup training by a factor of 3.25 (when M=5); in this case, training data was being read over a network, causing a significant IO bottleneck. Interestingly, this paper showed that larger batch sizes can support higher M-values. All three of these studies are focused on leveraging BR for the supervised training of CNN-based computer vision applications. In

this work, we are focused on leveraging BR for the *self-supervised* pretraining of *transformer-based* CV applications.

Numerous recent studies [24, 25, 26, 27, 31, 32] have leveraged BR to improve the performance on transformer-based CV applications; however, they do not report speedup times. Fuller et al. [29] pretrain a vision transformer on multi-spectral remote sensing imagery using masked autoencoding (MAE) [28] and leverage BR to speedup pretraining by around 6 times (with M=8); however, they do not report the performance of a pretraining run that does not use BR with which to compare. Girdhar et al. [30] pretrain vision and video transformers using MAE and explore M {1, 2, 4, 8}. They demonstrate that all M-values result in approximately the same model accuracy for both vision and video models. For vision models, they report a speedup factor of around 1.05; for video models, they report a speed factor of around 1.2. These speedup factors are much lower than [29] or [23] because they have access to state-of-the-art hardware (i.e., Facebook AI's internal cluster) and optimized software that significantly reduces their data IO bottleneck in the baseline case. Their video processing models see higher speedups because their video data is larger and takes longer to load than their vision data. As we can see, the speedup factors—due to BR—are primarily a function of hardware and the amount of data being loaded. In this work, we will report the performance based on the ratio of model training to total wall-clock time (including data IO)—this is often referred to as accelerator (GPU or TPU) utilization. Thus, our results will be valuable to all infrastructures and applications.

2.4 Basics of TPUs

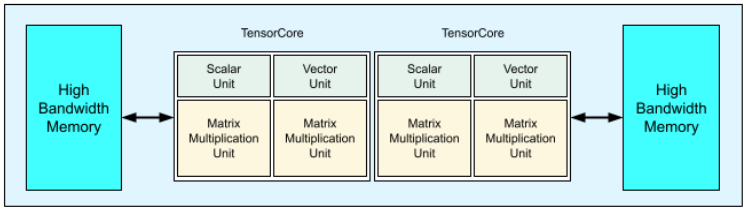

Figure 4: Simplified 3rd generation Tensor Processing Unit (TPU) chip diagram. This figure is taken from Google's documentation [33].

Google offers free TPUs to researchers via their TPU Research Cloud (TRC) program [34]. The TRC program provides access to second and third-generation TPU virtual machines with 8 cores; thus, each VM is called a TPUv2-8 (second-generation) or TPUv3-8 (third-generation). The main difference between generations is the number of MXUs per core; second generation cores have 1 MXU, whereas third generation cores have two MXUs (see figure 4). (The cores also come with a scalar unit and a vector unit.) This means that a TPUv3-8 VM contains 16 MXUs, and a TPUv2-8 VM contains 8 MXUs. Each MXU is a 128x128 systolic array, corresponding to 16K multiply-accumulate operations per clock cycle.

3 Problem Statement

1. (a) Can I use multi-core TPUs to speedup DNN training? (b) Can I use BR to further speedup DNN training? (c) If BR results in worse model performance, can I recover

it in a second stage of training?

2. (a) Although BR has been extensively used, BR multiples greater than 8 have not been studied in the context of self-supervised training of ViTs. (b) It is unclear if any performance decreases—due to BR—can be quickly recovered in a second stage of training.

3. Historically, DNN training can be performed with very high accelerator utilization; this makes BR unnecessary. But this is changing. (i) Faster accelerators (e.g., GPUs or domain-specific hardware like TPUs) and accelerator algorithms decrease accelerator utilization (as measured by the fraction wall-clock time). (ii) More data (resulting in better model performance) that does not fit in host memory or high-speed storage, and must be stored on slower storage devices, decreases accelerator utilization. (iii) Different data types (e.g., hyperspectral, video, 3D data) take longer to load from disk and decrease accelerator utilization. These three trends worsen the data IO bottleneck that BR attempts to address.

4 Proposed Solution

For this work, I will be using a dataset I collected of 1 million remote sensing images. These images are stacked multi-spectral optical and synthetic aperture radar imagery—stacked along the channel dimension. Each image has a shape of {15, 112, 112}, where 15 is the number of channels, and 112 is the height and width of each image. My data is stored in 16-bit floats and consumes roughly 360 GB of storage. (I also collected 10k images from regions of the world not present in the training dataset, this is our test set.) Since I am training on Google's TPUs, I needed to store my data in the same zone as the TPUs (in my case, "europe-west4-a"). Just like MAE's original setting, I will use a mask ratio of 75% and train for 204k steps with a batch size of 1024—thus, all models will be trained on roughly 208 million samples. All models will use the ViT-Base [16] architecture, which is composed of roughly 90 million learnable parameters distributed over 12 transformer layers.

4.1 Training DNNs with multi-core TPUs

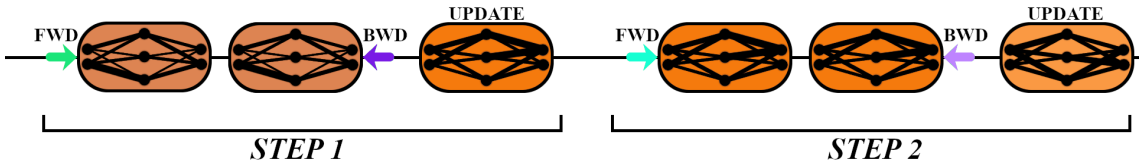

Figure 5: A model (depicted by a tiny feedforward neural network) being updated over 2 steps, on a single device.

My first step is to set up DNN training on multi-core TPUs. In practise, training DNNs on more than one device (TPU core or GPU, see figure 6) is mostly the same as training DNN models on a single device (see figure 5). For multi-device training—specifically distributed data-parallel training (DDP)—the DNN is first copied to each device. In parallel, each process loads data from storage, performs any data preprocessing, and runs the data forward through the DNN, then backward (calculating gradients); thus far, this is no different from

single-process DNN training. At this stage, gradients are averaged across the processes, and finally, the model parameters are updated. Because gradients are averaged across the processes, the models are still identical across devices. After this parameter update, each process loads the next batch of data, and all steps are repeated.

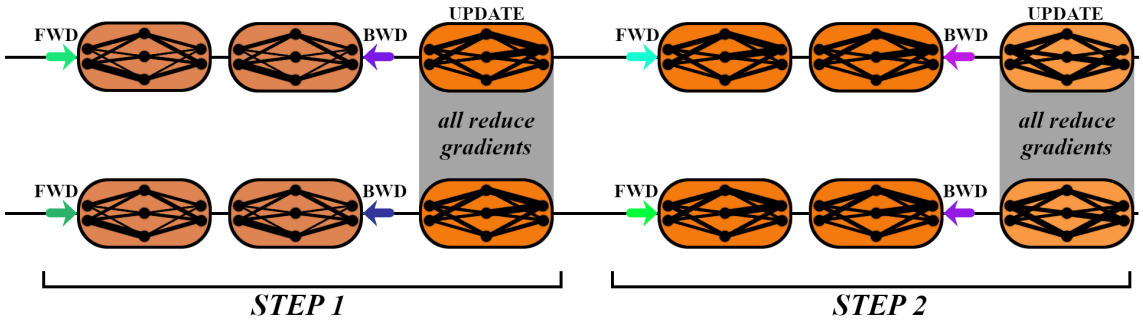

Figure 6: A model (depicted by a tiny feedforward neural network) being updated over 2 steps, on *two* devices. Each device gets a different batch of data for each step (depicted by slight variations in color).

I used PyTorch XLA (Accelerated Linear Algebra), a deep learning compiler, to implement this. Programatically there are only a few differences between training on a single device and multiple devices in parallel. One, I wrap my PyTorch model in XLA's multiprocessing model wrapper:

$$torch_xla.distributed.xla_multiprocessing.MpModelWrapper(model)$$

Two, I spawn multiple instances (8 in my case) of my training script also using XLA:

$$torch_xla.distributed.xla_multiprocessing.spawn(train_function(), nprocs = 8)$$

Three, instead of calling PyTorch's default optimizer step—which updates the model parameters based on the gradients—I call XLA's optimizer step:

$$torch_xla.core.xla_model.optimizer_step(optimizer)$$

Behind the scenes, XLA's optimizer step adds the gradients across all cores (via all reduce), then divides by the number of cores, resulting in a mean operation. Although these are simple function calls, getting my application running on TPU VMs was very challenging, as I will discuss in more detail in section 5.2. Finally, the programmer must ensure that each core loads a different batch of data; otherwise, they are not leveraging multi-device training! Since each core processes a different batch of data, training with multiple devices increases the number of images per second, i.e., the throughput.

4.2 Batch Repetition

I will train 6 models—5 leveraging BR (sweeping M {2, 4, 8, 16, 32}), and the baseline that does not leverage BR. After training for 204k steps, I will perform a second stage of training by continuing to train each model without BR for just 5k steps. This second stage of training may allow me to recover some of the decrease in accuracy that BR causes.

5 Experimental Evaluation

5.1 Scaling TPU Cores

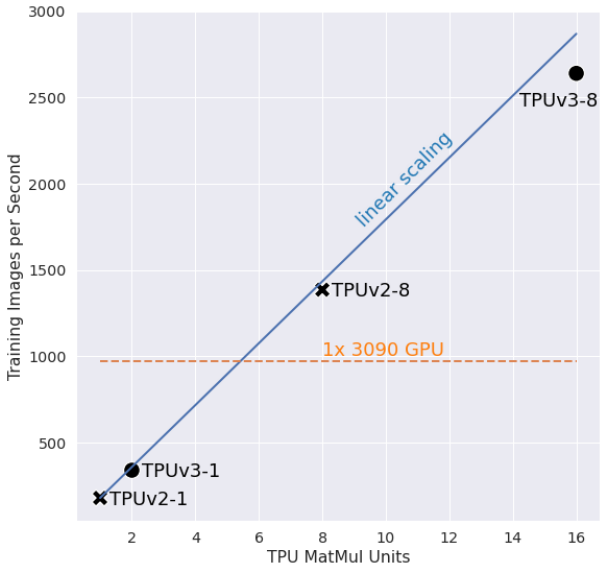

Figure 7: Scaling performance of TPU matrix multiplication units (MXUs) when training a vision transformer using masked autoencoding.

Looking at figure 7, the number of MXUs scales almost linearly with respect to training throughput. This aligns with Google's claims [13] that show almost linear scaling up to 512 MXUs (256 TPUv3 cores). Although, to my knowledge, this is the first time TPU scaling performance has been demonstrated using vision transformers (previous scaling results were demonstrated using CNNs and MLPs). I plotted both generations of TPUs together for easy comparison, but I will calculate their exact scaling performance separately.

TPUv2 scale with 96.6% efficiency, from 1 to 8 cores. TPUv3 scale with 97.1% efficiency, from 1 to 8 cores. TPUs cannot run with core counts between 1 and 8, so I could not get more data points to show the scaling properties when the number of cores is between 1 and 8. Additionally, I was surprised to see even the previous generation of TPUs (i.e., TPUv2) outperform an RTX 3090 GPU by 42.7%. In other words, 8 TPU MXUs outperform an RTX 3090's 328 tensor cores for my application.

5.2 Batch Repetition Speedup

Batch Rep.	Test Loss (x10^{-2})			Speedup (w/ util)		
M	102k steps	204k steps	+5k steps*	10%	50%	90%
1	4.78	4.63	4.51	-	-	-
2	4.84	4.66	4.54	1.82	1.33	1.05
4	4.93	4.75	4.60	3.08	1.60	1.08
8	5.07	4.82	4.67	4.71	1.78	1.10
16	5.34	5.06	4.81	6.40	1.88	1.10
32	5.57	5.17	4.98	7.80	1.94	1.11

Table 1: Test set loss across BR multiples, along with speedup factors for reference. *

indicates a second stage of training—for an additional 5k steps—that does not include any data augmentation or BR.

As we can see from table 1, there is a clear trade-off between speedup and model accuracy. But this is in contrast to prior work that demonstrates no trade-off, at least up to M=8. I believe the reason for this is data augmentation. Imagine using BR without any data augmentation; this would amount to repeating identical copies of samples M times. Identical samples in a batch result in identical gradients, providing no additional information for the model to learn. But using data augmentation increases the within-batch variance—effectively simulating new samples—that *can* result in additional information for the model to learn. If the data augmentation is sub-optimal—either so strong that the simulated samples are no longer realistic or so weak that it does not increase the within-batch variance enough—then BR may result in worse model accuracy. This is what happened in my experiments, i.e., my choice of data augmentation was sub-optimal.

My data augmentation was sub-optimal because I could not employ standard data augmentation methods such as flipping, cropping, and rotating samples. When training on a TPU with XLA [35], XLA optimizes the computation graph at the start of training—this slowly ramps up the throughput over around 10 minutes (from my experience). When I implemented standard data augmentation methods—that I've used extensively on GPUs—XLA could not optimize the computation graph resulting in very low throughput. But since I was not given clear error messages indicating the source of the problem, I eliminated augmentations one-by-one. I found that XLA could optimize only horizontal and vertical flipping. And when I implemented BR, XLA could not even optimize the computation of these augmentations either. Therefore, the only augmentations I could add was random Gaussian noise that only required a simple element-wise addition to the input tensors. Before running my experiments, I anticipated this would seriously limit my results, but I could not find a fix.

Comparing my code to examples provided by torch-XLA [36], the only difference was that I needed to augment images that were PyTorch tensors. In contrast, the torch-XLA examples would augment images that were jpegs, *then* convert the augmented samples to PyTorch tensors. I did not convert my data to 8-bit integers or jpegs, because higher precision data is important for my application. This, I believe, is the source of XLA's inability to properly optimize my training runs when using data augmentation.

That all being said, even with my experimental limitations, M=2 (test loss of 0.0466) still performed *very* close to my baseline (test loss of 0.0463). When the accelerator utilization is 50% (which was close to my actual TPU utilization using a local SSD to read data from), this would reduce the costs of training by 33%—this is significant.

Since these results—i.e., the trade-off between computational speedup and model accuracy—were not as good as I had hoped; I attempted to recover the decrease in model accuracy by adding another stage of training. This second stage of training does not include any data augmentation or BR and is only run for 5k steps (compared to 204k steps in the first stage of training). This stage was able to recover much of the performance gap when using higher M values. For instance, after this stage of training, the M=4 model outperformed (test loss of 0.0460 versus 0.0463) the baseline without this additional stage of training. As a result, I was able to keep most of the speedup gains while improving the model's accuracy on a held-out test set.

6 Conclusions

6.1 Multi Device Training with TPUs

The computational resources required to train state-of-the-art DNNs has grown exponentially since the field's inception. This has led researchers and practitioners to train their models with multiple devices in parallel to speedup training. Additionally, companies have started designing domain-specific hardware to train their models more efficiently.

In this work, I implemented a state-of-the-art self-supervised learning algorithm (masked autoencoding) on multi-core TPUs (both TPUv2 and TPUv3)—these machines were designed to efficiently train DNNs. Since they are multi-core devices, this requires the use of distributed model training that reduces gradients across devices before updating model parameters—this step is unique to training models on multiple devices in parallel.

6.2 Batch Repetition

BR is a method that reuses data loaded from storage that reduces data IO time, and makes better use of AI accelerators such as GPUs or TPUs. As the computation required to train state-of-the-art DNNs continue to increase and the accelerators, themselves, continue to improve, BR usage will continue to increase.

In this work, I implemented BR on TPUv3 machines that resulted in significant speedups, depending on BR multiples. However, I was unable to implement optimal data augmentation techniques that are essential to BR. As a result, I found a trade-off between speedup and model accuracy that I do not expect if optimal data augmentation was used. As a consolation, I was able to recover much of the decrease in model accuracy but introducing a second stage of training—for just 5k additional learning steps. This is a significant result because I was able to demonstrate that I could speedup model training without hurting model accuracy.

I believe I could not implement optimal data augmentation methods because my data were stored as PyTorch tensors (16-bit floats) rather than jpegs; this is the only difference between my code and torch-xla examples. In future work, I will try converting my data to jpegs (losing some precision) or find another image type that retains precision but can still be optimized by XLA on TPUs. I believe this will address the limitations of this work.

References

[1] Chakradhar, Srimat and Sankaradas, Murugan and Jakkula, Venkata and Cadambi, Srihari. "A Dynamically Configurable Coprocessor for Convolutional Neural Networks", Proceedings of the 37th Annual International Symposium on Computer Architecture, 2010.

[2] Lopes, Noel and Ribeiro, Bernardete and Gonçalves, João. "Restricted Boltzmann Machines and Deep Belief Networks on multi-core processors", International Joint Conference on Neural Networks (IJCNN), 2012.

[3] Bhuiyan, Mohammad A. and Pallipuram, Vivek K. and Smith, Melissa C. and Taha, Tarek and Jalasutram, Rommel. "Acceleration of spiking neural networks in emerging multi-core and GPU architectures", IEEE International Symposium on Parallel Distributed Processing, Workshops and Phd Forum (IPDPSW), 2010.

[4] Dolan, Ryanne and DeSouza, Guilherme. "GPU-based simulation of cellular neural networks for image processing", International Joint Conference on Neural Networks, 2009.

[5] Jang, Honghoon and Park, Anjin and Jung, Keechul. "Neural Network Implementation Using CUDA and OpenMP", Digital Image Computing: Techniques and Applications, 2008.

[6] Bhuiyan, Mohammad A. and Jalasutram, Rommel and Taha, Tarek M. "Character recognition with two spiking neural network models on multi-core architectures", IEEE Symposium on Computational Intelligence for Multimedia Signal and Vision Processing, 2009.

[7] Krizhevsky, Alex and Sutskever, Ilya and Hinton, Geoffrey E. "ImageNet Classification with Deep Convolutional Neural Networks", Proceedings of the 25th International Conference on Neural Information Processing Systems, 2012.

[8] Lecun, Y. and Bottou, L. and Bengio, Y. and Haffner, P. "Gradient-based learning applied to document recognition", Proceedings of the IEEE, 1998.

[9] Kaplan, Jared and McCandlish, Sam and Henighan, Tom and Brown, Tom B. and Chess, Benjamin and Child, Rewon and Gray, Scott and Radford, Alec and Wu, Jeffrey and Amodei, Dario. "Scaling Laws for Neural Language Models", arxiv, 2020.

[10] Hoffmann, Jordan and Borgeaud, Sebastian and Mensch, Arthur and Buchatskaya, Elena and Cai, Trevor and Rutherford, Eliza and Casas, Diego de Las and Hendricks, Lisa Anne and Welbl, Johannes and Clark, Aidan and Hennigan, Tom and Noland, Eric and Millican, Katie and Driessche, George van den and Damoc, Bogdan and Guy, Aurelia and Osindero, Simon and Simonyan, Karen and Elsen, Erich and Rae, Jack W. and Vinyals, Oriol and Sifre, Laurent. "Training Compute-Optimal Large Language Models", Advances in Neural Information Processing Systems, 2022.

[11] Zhai, Xiaohua and Kolesnikov, Alexander and Houlsby, Neil and Beyer, Lucas. "Scaling Vision Transformers", IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2022.

[12] Abnar, Samira and Dehghani, Mostafa and Neyshabur, Behnam and Sedghi, Hanie. "Exploring the Limits of Large Scale Pre-training", International Conference on Learning Representations (ICLR), 2022.

[13] Jouppi, Norman P. and Yoon, Doe Hyun and Kurian, George and Li, Sheng and Patil, Nishant and Laudon, James and Young, Cliff and Patterson, David. "A Domain-Specific Supercomputer for Training Deep Neural Networks", Communications of the ACM, 2020.

[14] Norrie, Thomas and Patil, Nishant and Yoon, Doe Hyun and Kurian, George and Li, Sheng and Laudon, James and Young, Cliff and Jouppi, Norman and Patterson, David. "The Design Process for Google's Training Chips: TPUv2 and TPUv3", IEEE Micro, 2021.

[15] Vaswani, Ashish and Shazeer, Noam and Parmar, Niki and Uszkoreit, Jakob and Jones, Llion and Gomez, Aidan N. and Kaiser, Lukasz and Polosukhin, Illia. "Attention Is All You Need", Advances in Neural Information Processing Systems (NeurIPS), 2017.

[16] Dosovitskiy, Alexey and Beyer, Lucas and Kolesnikov, Alexander and Weissenborn, Dirk and Zhai, Xiaohua and Unterthiner, Thomas and Dehghani, Mostafa and Minderer, Matthias and Heigold, Georg and Gelly, Sylvain and Uszkoreit, Jakob and Houlsby, Neil. "An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale", International Conference on Learning Representations (ICLR), 2021.

[17] Deng, Jia and Dong, Wei and Socher, Richard and Li, Li-Jia and Kai Li and Li Fei-Fei. "ImageNet: A large-scale hierarchical image database", IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2009.

[18] Fuller, Anthony and Millard, Koreen and Green, James R. "SatViT: Pretraining Transformers for Earth Observation", IEEE Geoscience and Remote Sensing Letters, 2022.

[19] Zhang, Renrui and Guo, Ziyu and Gao, Peng and Fang, Rongyao and Zhao, Bin and Wang, Dong and Qiao, Yu and Li, Hongsheng. "Point-M2AE: Multi-scale Masked Autoencoders for Hierarchical Point Cloud Pre-training", Advances in Neural Information Processing Systems (NeurIPS), 2022.

[20] Tong, Zhan and Song, Yibing and Wang, Jue and Wang, Limin. "VideoMAE: Masked Autoencoders are Data-Efficient Learners for Self-Supervised Video Pre-Training", Advances in Neural Information Processing Systems (NeurIPS), 2022.

[21] Hoffer, Elad and Ben-Nun, Tal and Hubara, Itay and Giladi, Niv and Hoefler, Torsten and Soudry, Daniel. "Augment Your Batch: Improving Generalization Through Instance Repetition", Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), June 2020.

[22] Fort, Stanislav and Brock, Andrew and Pascanu, Razvan and De, Soham and Smith, Samuel L. "Drawing Multiple Augmentation Samples Per Image During Training Efficiently Decreases Test Error", arXiv, June 2021.

[23] Choi, Dami and Passos, Alexandre and Shallue, Christopher J. and Dahl, George E. "Faster Neural Network Training with Data Echoing", arXiv, July 2019.

[24] Touvron, Hugo and Cord, Matthieu and Douze, Matthijs and Massa, Francisco and Sablayrolles, Alexandre and Jegou, Herve. "Training data-efficient image transformers amp; distillation through attention", Proceedings of the 38th International Conference on Machine Learning (ICML), July 2021.

[25] Bao, Hangbo and Dong, Li and Piao, Songhao and Wei, Furu. "BEiT: BERT Pre-Training of Image Transformers", International Conference on Learning Representations (ICLR) 2022.

[26] Feichtenhofer, Christoph and Fan, Haoqi and Li, Yanghao and He, Kaiming. "Masked Autoencoders As Spatiotemporal Learners", arXiv, May 2022.

[27] Xie, Jiahao and Li, Wei and Zhan, Xiaohang and Liu, Ziwei and Ong, Yew Soon and Loy, Chen Change. "Masked Frequency Modeling for Self-Supervised Visual Pre-Training", arXiv, June 2022.

[28] He, Kaiming and Chen, Xinlei and Xie, Saining and Li, Yanghao and Dollár, Piotr and Girshick, Ross. "Masked Autoencoders Are Scalable Vision Learners", Proceedings

of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), June 2022.

[29] Fuller, Anthony and Millard, Koreen and Green, James R. "Transfer Learning with Pretrained Remote Sensing Transformers", arXiv, September 2022.

[30] Girdhar, Rohit and El-Nouby, Alaaeldin and Singh, Mannat and Alwala, Kalyan Vasudev and Joulin, Armand and Misra, Ishan. "OmniMAE: Single Model Masked Pretraining on Images and Videos", arXiv, June 2022.

[31] Geiping, Jonas and Goldblum, Micah and Somepalli, Gowthami and Shwartz-Ziv, Ravid and Goldstein, Tom and Wilson, Andrew Gordon. "How Much Data Are Augmentations Worth? An Investigation into Scaling Laws, Invariance, and Implicit Regularization", arXiv, October 2022.

[32] Song, Yuxin and Yang, Min and Wu, Wenhao and He, Dongliang and Li, Fu and Wang, Jingdong. "It Takes Two: Masked Appearance-Motion Modeling for Self-supervised Video Transformer Pre-training", arXiv, October 2022.

[33] https://cloud.google.com/tpu/docs/system-architecture-tpu-vm

[34] TPU Research Cloud: https://sites.research.google/trc/about/

[35] Sabne, Amit. "XLA : Compiling Machine Learning for Peak Performance", 2020

[36] PyTorch/XLA: https://github.com/pytorch/xla